

PSSAR Applied Research · Data & ML Bootcamp

Capstone Project Brief

Format: solo · **Deliverable:** a defensible model behind a working, versioned API

1. What you are building

Every project — whatever topic you pick — produces the **same kind of deliverable**: a machine-learning model you can *defend*, wrapped in a **versioned REST API** that returns predictions.

This brings the whole course together in one artefact. You will pick a **baseline**, compare **at least two models**, choose the **right metric** (not bare accuracy), check for **leakage**, **tune** your finalist, wrap it in a **validated API**, **version** it, and write a **conclusion-first report**.

Accuracy is not the point. A modest model, honestly evaluated, cleanly served, and clearly explained is worth far more than a flashy number with no methodology. What matters is how you *work*, not how lucky your dataset was.

2. What you must deliver

1. **A clean, well-organised repo** pushed to GitHub, with code and model artefacts separated.
2. **A conclusion-first `README.md`** — your recommendation and its key number in the **first two sentences**, before any method.
3. **A defensible model**: baseline → ≥ 2 models compared → right metric → no leakage → tuned finalist.
4. **A debugging log**: at least one real problem, written up as *Symptom* → *Diagnosis* → *Fix*.
“Nothing was wrong” is not acceptable.
5. **A working, versioned API** exposing `/health`, `/predict` (with input validation), `/metadata`, and `/versions`. It must **run and return predictions** (running locally is fine), with **≥ 2 model versions**, one promoted live, and the version returned in every prediction.

6. **Reproducibility:** pinned `requirements.txt` , a metadata file per model version, fixed random seeds, and a short model card. Your repo should pass the **clone test** (§6).

Optional extra: deploy the API to a **public URL** (e.g. a free host) so others can call it from anywhere. This is a nice bonus, **not** a requirement — a validated API that runs locally satisfies deliverable #5 fully.

3. Choose a project

The datasets below are optional suggestions. You may choose your own dataset and topic — as long as your project meets all the deliverables in section #2. The list exists to save you time and to show the range of what fits; it is not a menu you're restricted to.

Prefer datasets whose **loading is scripted** (your code fetches/reads it), so your work stays reproducible; avoid anything that needs manual clicking or steps a classmate can't repeat.

All suggested sources below were checked and load without a login. UCI datasets load in one line with the `ucimlrepo` package: `pip install ucimlrepo` then `from ucimlrepo import fetch_ucirepo; d = fetch_ucirepo(id=222)` . The Telco CSV loads directly with `pandas.read_csv(<raw URL>)` . `sklearn` / `torchvision` / `HF` datasets load via their own functions.

Tabular — classical ML

Project	What you actually do	Suggested dataset
Customer churn	Predict which telecom customers cancel. Imbalanced; pick a threshold; explain the precision/recall trade.	<code>raw.githubusercontent.com/IBM/telco-customer-churn-on-icp4d/master/data/Telco-Customer-Churn.csv</code>
House-price estimator	Regression: predict median house value. Regression metrics (RMSE/MAE), residual analysis.	<code>sklearn.datasets.fetch_california_housing</code>
Loan-default risk	Imbalanced classification: will a borrower default? Threshold choice,	UCI Default of Credit Card Clients

Project	What you actually do	Suggested dataset
	cost trade-offs.	archive.ics.uci.edu/dataset/350/default+of+credit+card+clients
Disease / readmission risk	Imbalanced health classification; the “accuracy lies” lesson made real.	UCI Diabetes 130-US Hospitals archive.ics.uci.edu/dataset/296/diabetes+130-us+hospitals+for+years+1999-2008
Demand / delay predictor	Tabular + engineered time features (hour, day, season). Feature engineering is the point.	UCI Bike Sharing archive.ics.uci.edu/dataset/275/bike+sharing+dataset
Bank term-deposit (<i>course dataset</i>)	Reuse the bank-marketing data end-to-end: the leakage trap, imbalance, serving.	UCI Bank Marketing archive.ics.uci.edu/dataset/222/bank+marketing
Forest cover type	Multi-class tabular classification (7 forest types). Handling many classes and a large table.	<code>sklearn.datasets.fetch_covtype</code>
NN vs gradient boosting on tabular	Build & tune an MLP (with categorical embeddings) and compare it to gradient boosting on the same table. Where NNs help on tabular — and where they don’t.	any tabular set above (bank / churn)

Vision — the network is your playground

These use a **free cloud GPU**. Cap image size and dataset size so a sweep finishes in minutes. The weighting rewards **what your experiments showed**, not the final accuracy — see the [△](#) note.

Project	What you actually do	Suggested dataset
MLP vs CNN architecture study	Build both from scratch ; vary hidden layers, width, activations; <i>measure</i> the gap and explain it via inductive bias.	<code>torchvision.datasets.FashionMNIST</code> / <code>MNIST</code>
CNN tuning lab	Train a small CNN; sweep depth, data augmentation, batch-norm, LR schedule; document what each knob did.	<code>torchvision.datasets.CIFAR10</code>
From-scratch vs fine-tuned bake-off	Train a small CNN yourself and fine-tune a pretrained ResNet on the same data; compare accuracy, training time, and data needed.	CIFAR-10, or <code>mohanty/PlantVillage</code> (HF) subset
Fine-tune an image classifier	Transfer learning for results: swap the head of a pretrained model, fine-tune on a small domain set.	<code>mohanty/PlantVillage</code> (HF <code>datasets</code>)
Traffic-sign recognition	Multi-class CNN on 43 sign classes; augmentation matters a lot here.	<code>torchvision.datasets.GTSRB</code>
Audio: spoken-digit / sound	CNN on spectrograms — a grid that isn't a photo. Convert audio → mel-spectrogram → classify.	Free Spoken Digit Dataset <code>github.com/Jakobovski/free-spoken-digit-dataset</code>

⚠ **Vision note.** *Playing with architecture and maxing accuracy* are different goals. Your **experiments** carry the weight — a documented sweep table (“depth 2→4 improved val by X; augmentation did Y; wider layers did nothing”) counts for **more** than the leaderboard number. A from-scratch CNN tuned to 82% with every knob explained is stronger than a fine-tune to 95% explained by nothing. Serve your best model behind the same API as everyone else.

Text & other

Project	What you actually do	Suggested dataset
News-topic / sentiment	Classical TF-IDF + linear baseline, then optionally fine-tune DistilBERT. Baseline-vs-transformer comparison.	HF datasets : fancyzhx/ag_news or stanfordnlp/imdb
Spam detection	Text classification with a strong classical baseline (TF-IDF + logistic/NB). Precision/recall trade on imbalanced text.	HF datasets : ucirvine/sms_spam
Live-data model	Pull data from a public API, model it, serve predictions. Exercises the “consuming external APIs” workflow fully.	Open-Meteo api.open-meteo.com/v1/forecast (no key) · CoinGecko api.coingecko.com/api/v3 · GitHub api.github.com

Researcher-shaped (Gonna leave these ideas here cause they are interesting -Hend)

Project	What you actually do
Leakage hunt	Take a public notebook reporting a suspiciously high score, find the leakage , fix it, and re-report the honest number. Serve the corrected model.
Replication study	Reproduce a published or public result from scratch, document every gap between what was claimed and what you got, and critique it.

4. Suggested steps

Not fixed. Adjust to your project.

1. **Frame it.** Pick project + data. Explore. Define the target **and the metric**. Ship a baseline.
2. **Compare.** Build the pipeline; train ≥ 2 models; pick a finalist.
3. **Experiment.** Tabular: tune the finalist. Vision: run your architecture / knob sweeps.
4. **Diagnose.** Error analysis; **find & fix at least one bug** and record it in the debugging log.

5. **Freeze & write up** what each experiment showed.
6. **Wrap** the model in the validated API; make the tests pass.
7. **Version** it (≥ 2 versions); promote one; (optionally) deploy to a public URL.
8. **Communicate**. Conclusion-first README; polish the repo; fill the model card.
9. **Peer-check**. Have a classmate run your repo cold; fix whatever they couldn't; prep your demo.
10. **Buffer**. Something always slips — leave room.

5. How do we look at the work

These are the areas the project emphasises:

- Sound methodology / no leakage
- Right metric + honest evaluation
- Model choice + baseline + tuning / experiments
- Working, validated API
- Versioning + reproducibility
- Communication (conclusion-first README + demo)

6. Definition of done

- Public GitHub repo, code and model artefacts separated.
- `README.md` recommendation is in the **first two sentences**.
- Baseline + ≥ 2 models + the metric justified in writing.
- Debugging log has ≥ 1 real Symptom→Diagnosis→Fix entry.
- API runs and returns predictions: `/health` , `/predict` (bad input → 422), `/metadata` , `/versions` .
- ≥ 2 model versions; one promoted; `model_version` returned in every prediction.
- Pinned `requirements.txt` ; tests pass.
- **Clone test**: a classmate clones your repo and gets a prediction using only your README. **If possible** Whatever they have to ask you is what's missing — fix it.
- (Optional bonus) a live public URL anyone can call.

7. Scope guardrails

- **Good signs** — your data-loading is **scripted and reproducible** (your code fetches or reads it, not manual downloads someone else can't repeat); the effort goes into method, experiments, and serving, not days of wrangling.
- **Too big (don't)**: collecting your own data from scratch · anything past a free cloud GPU · training a large model from scratch · building a web/mobile frontend · real-time streaming.
- **Too small (don't)**: a notebook with no API · a toy dataset hitting 99% with nothing evaluated honestly and nothing debugged.
- **Vision-specific**: downscale images and/or subsample so a 6-config × 10-epoch sweep fits one cloud session. A documented sweep on a subset beats one heroic full run. (numbers are examples, it depends on your dataset I think)

8. FAQ

- **Do I have to use one of the listed datasets?** No. **The lists are optional suggestions — bring your own dataset and topic if you prefer, as long as it meets every deliverable in section #2.**
- **Do I have to deploy to a public URL?** No — that's an optional bonus. Your API only needs to run and return validated predictions; running locally is enough.
- **My model is barely better than the baseline.** That can be a *strong* project if you show it honestly and explain why. A well-argued negative result is real research.
- **Do I need a GPU?** Only the vision options and the DistilBERT text option. A free cloud GPU is enough. Everything else runs on a laptop. Don't forget and run everything on the GPU, some things can be done on the laptop itself.
- **Can I pick a topic that isn't listed at all?** Yes, if it fits the guardrails and you clear it with me. It must still produce the standard deliverable.
- **How “good” does the model have to be?** There is no accuracy threshold. The emphasis (see section #5) is on method, honesty, a working versioned API, and clear communication.
- **What if serving the model eats my last stretch of time?** Get a placeholder model behind the API early, so by the time your real model is ready you're only swapping a file.